

Toward a Theory of Computation for Learned Transformer Machines: Tasks, Resources, Learnability, Composition, and Closure

Paper 0.1 — Position and Formal Framework

Draft inception — August 2026

Abstract

The dominant theoretical question about large language models is often phrased too coarsely: “what can a Transformer compute?” Classical expressivity results characterize idealized Transformers through formal languages and circuit classes, while empirical work usually asks whether a trained model succeeds on a benchmark. Between these views lies a missing object: the computational theory of a *learned* Transformer machine. A computation may be representable by an architecture yet rarely acquired by training; it may be learned on bounded instances yet fail to compose; it may compose within one forward pass but fail to close over unbounded instance depth; autoregressive generation, in-context information, or an external harness may alter these frontiers in different ways. We propose a discrete framework separating six resources: atomic operator/syntax complexity, internal one-pass composition, autoregressive multi-token composition, closure, in-context information, and external harness/tool execution. For each task family we distinguish information sufficiency, representability, acquisition probability, coverage, systematic executability, and closure. The program is intended as an empirical analogue of automata and complexity theory for learned neural machines: tasks are classified by the state and iteration they require, machines by the resources they expose, and experiments estimate competence and acquisition frontiers rather than infer algorithms from endpoint accuracy. Papers 0.5–0.9 progressively instantiate this framework with predictive patterns, semantic relations, formal inference and unification, contextual computation, autoregressive recurrence, and explicit iterative harnesses.

1 Why a learned-machine theory is needed

A fixed architecture can represent computations that gradient-based training rarely discovers. A trained model can fit finite examples without acquiring a reusable rule. A rule can be reusable over a bounded range without supporting closure. Conversely, autoregressive generation can repeatedly apply the same learned local transformation and thereby change the effective computational machine without changing the model weights.

We therefore separate:

representability $\neq$ learnability $\neq$ reliable acquisition $\neq$ systematic executability $\neq$ closure.

Classical Transformer theory provides essential upper and lower bounds under explicit architectural and precision assumptions. Formal-language work relates fixed-depth attention models to circuit classes such as AC^0 and TC^0 , while recent chain-of-thought theory shows that autoregressive intermediate computation can substantially expand expressivity. Our objective is complementary: characterize the *trained finite machine* and measure how its empirical computational frontier depends on architecture, training distribution, generated state, context, and external execution resources.

2 Basic objects

2.1 Task family

A task family is

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, \mathcal{D}, \mathcal{R}),$$

where $\mathcal{D}$ is a generator/distribution and $\mathcal{R}$ is the declared structural prior that makes prediction possible.

Each instance has a complexity vector

$$\mathbf{C} = (S, K, B, V, N, M, \dots),$$

where S is atomic syntax/operator complexity, K required composition/proof depth, B branching/search width, V variable/state complexity, N nuisance context, and M memory/context requirement. Complexity is generally partially ordered, so measured Pareto competence regions are preferable to a single scalar difficulty.

2.2 Information sufficiency

Before studying an intelligent mechanism, determine whether the task is information-theoretically solvable. For query X , context D , and target Y :

$$I(Y; D | X) = H(Y | X) - H(Y | X, D).$$

For deterministic rule inference under declared prior $\mathcal{R}$, require

$$H(Y | X, D, \mathcal{R}) = 0.$$

An arbitrary unseen random mapping with insufficient demonstrations is not a reasoning problem: no oracle lacking additional information can exceed the Bayes ceiling.

Every task release should therefore include an information-validity record

$$\mathcal{I} = (H(Y | X), H(Y | X, D), I(Y; D | X), \mathcal{R}, A_{\text{Bayes}}, |\mathcal{H}_{X,D,\mathcal{R}}|, m_{\text{id}}),$$

where $\mathcal{H}_{X,D,\mathcal{R}}$ is the set of hypotheses consistent with the query, demonstrations, and declared prior, and m_{id} is the minimum context needed for identifiability. Entropies are distribution-dependent quantities, not properties of a serialization alone. If the generator is deterministic only after conditioning on $\mathcal{R}$, that conditioning must be stated rather than smuggled into an oracle.

Proposition 1 (Information impossibility). *Under zero-one loss, if $H(Y | X, D, \mathcal{R}) > 0$, then no predictor measurable with respect to $(X, D, \mathcal{R})$ is universally correct. Its optimal expected accuracy is*

$$A_{\text{Bayes}} = \mathbb{E}_{X,D,\mathcal{R}} \max_y P(Y = y | X, D, \mathcal{R}) < 1$$

whenever the conditional distribution is non-degenerate on a set of positive measure.

Pointwise, the best decision selects the most probable conditional label; positive conditional entropy entails ambiguity on at least one event of positive measure. This separates model failure from task underdetermination.

2.3 Machine hierarchy

M0: one-pass / one-token Transformer.

$$Y = F_{\theta}(X).$$

M1: autoregressive Transformer.

$$y_t = F_{\theta}(X, y_{<t}).$$

Generated tokens are writable external state visible through self-attention.

M2: structured AR state. The same Transformer emits an explicit state trajectory with typed/constrained fields but no external semantic operation.

M3: deterministic proto-tool. The model emits an operation request; an external primitive evaluates it and appends a result.

M4: Transformer plus iterative harness. A loop repeatedly invokes the model until a learned or explicit stop condition.

M5: richer agent machine. Multiple tools, external memory, retrieval, search, or sub-agents. This is downstream and should not be invoked before simpler classes are understood.

The hierarchy is a resource ordering, not a theorem of strict containment without interface assumptions. M1 can emulate a structured trace if it reliably learns the format; M2 changes the admissible state language and often the training problem, not necessarily ideal expressivity. M3 adds a semantic transition computed outside the network. M4 adds invocation, state persistence, and termination control. Each comparison should record

$$\Delta\mathcal{M} = (\Delta\text{information}, \Delta\text{writable state}, \Delta\text{semantic primitives}, \Delta\text{control}, \Delta\text{training}),$$

preventing an easier prompt, a more reliable optimizer, and a stronger machine from all being called “augmentation.”

3 Six computational axes

3.1 Axis 1: atomic syntax and local operator learning

What is the most complex single-step relation a model of size (L, W, H) learns reliably? A rough ladder is

$$\begin{aligned} &\text{association} \rightarrow \text{n-gram} \rightarrow \text{unary} \rightarrow \text{binary} \rightarrow \text{Boolean rule} \\ &\rightarrow \text{predicate+variables} \rightarrow \text{nested functor} \rightarrow \text{type dispatch} \rightarrow \text{rewrite}. \end{aligned}$$

Define acquisition probability

$$P_{\text{acquire}}(\tau; M, \mathcal{D}, T) = P_s[A_s(\tau) \geq \gamma]$$

and coverage over the intended legal instance set. The falsifiable resource question is whether a controlled change in width, heads, layers, data, or budget moves this boundary across seeds. A single successful seed establishes neither learnability nor reliable acquisition.

3.2 Axis 2: internal one-pass composition

Assume local operator f is competent. How much composition can one forward pass implement?

$$x \mapsto f(x) \mapsto f^2(x) \mapsto \cdots \mapsto f^K(x).$$

Define

$$K_{\text{internal}}(M, \tau) = \max\{K : A(f^K; M) \geq \gamma\}.$$

A larger frontier does not prove serial layer-by-layer execution. Required dependency radius, operator count, proof depth, branching, nuisance, and raw length must be crossed rather than conflated. Ordered intermediate decoding and causal transfer are required before an $L \times K$ interaction is interpreted as serial internal computation.

3.3 Axis 3: autoregressive multi-token composition

Autoregression exposes recurrence:

$$s_{t+1} = F_{\theta}(X, s_{\leq t}).$$

For a parent relation, a root trajectory may be serialized as

$$C \rightarrow B \rightarrow A \rightarrow R.$$

Self-attention can read the entire generated history, so the state may be a tuple, proof prefix, substitution, or counter. Define K_{AR} and compare it directly with K_{internal} . Report local transition accuracy, stopping accuracy, total-sequence success, generated steps, and train-short/test-long behavior. This distinguishes a competent transition with accumulated error from a failed recurrence controller.

3.4 Axis 4: closure

Distinguish:

$$\forall K < \infty \exists M_K$$

(bounded family solvability), from a fixed-model tested range

$$\exists M \forall K \leq K_{\max}^{\text{test}},$$

and from true algorithmic closure

$$\exists M \forall K < \infty.$$

The last cannot be established empirically, but closure signatures can be tested by train-short/test-long extrapolation, computation length scaling with instance complexity, local-transition reuse, and termination generalization. Finite context creates a separate practical memory boundary.

Definition 1 (Budgeted closure signature). *For a fixed trained machine m and explicit context/step budget $C_{\max}$, define*

$$\Sigma_{m, C_{\max}} = (K_{\text{train}}, K_{\text{test}}, K_{\text{pass}}, \rho_{\text{step}}, \rho_{\text{stop}}, g(K), f_{\text{fail}}),$$

where K_{pass} is the largest tested complexity passing the multi-seed gate, ρ_{step} and ρ_{stop} are local-transition and stopping reliability, $g(K)$ is observed computation-length scaling, and f_{fail} is the first classified failure. This is a finite empirical object, not algorithmic closure.

3.5 Axis 5: in-context information

ICL changes what facts, rules, or parameters are available without changing weights. Separate fact acquisition from procedure acquisition. Contextual tasks should be classified as copy, constraint completion, finite task selection, parametric rule induction, or compositional/procedural induction. Holding X fixed while varying D is essential. Even when $I(Y; D \mid X) > 0$, controls must distinguish copying, elimination, task selection, parameter estimation, and construction of a reusable procedure.

3.6 Axis 6: harness and tool augmentation

External execution may expand the machine class or merely improve reliability. Therefore compare:

$$\text{one-token} \rightarrow \text{free multi-token} \rightarrow \text{structured scratchpad} \rightarrow \text{proto-tool} \rightarrow \text{iterative harness}.$$

Without the autoregressive baseline, tool benefits can be misclassified as new computational power. The prediction is task specific: a deterministic primitive should selectively rescue tasks whose failed transition it implements, while a format-only scratchpad changes writable state and often reliability. Cross-task rescue without primitive relevance instead implicates information or optimization.

4 Task classes by required state

Class	Required computation	Examples
A: local mapping	$x \mapsto f(x)$	parent, successor, one rewrite
B: fixed composition	$f^k(x)$, fixed k	grandparent, fixed proof chain
C: termination closure	iterate until state predicate	root, normalize-to-fixed-point
D: variable-count iteration	$(x, k) \mapsto (f(x), k - 1)$	ancestor at input level k
E: multi-state recurrence	$(x_t, y_t) \mapsto (x_{t+1}, y_{t+1})$	same ancestor level, paired traversal
F: search / memory	frontier or visited state	lowest common ancestor, proof search
G: binding/substitution	environment σ	first-order unification
H: proof-state recurrence	rule application over state	theorem proving

Table 1: Illustrative task classes organized by required machine state.

5 A learned-computation phase diagram

For machine class $\mathcal{M}$, task family τ , complexity $\mathbf{C}$, and training process Π , define

$$A(\tau, \mathbf{C}; \mathcal{M}, \Pi), \quad P_{\text{acquire}}(\tau, \mathbf{C}; \mathcal{M}, \Pi).$$

Here Π is a distribution over initialization s_θ , minibatch/order s_b , generator s_g , and optimizer/hyperparameters h , not a single lucky run:

$$P_{\text{acquire}} = \Pr_{s_\theta, s_b, s_g, h} \{A \geq \gamma, A_{\text{ctrl}} < \gamma_{\text{ctrl}}\}.$$

For a tested domain $\Omega_{\tau, \mathbf{C}}$, coverage is the measured mass or fraction on which the gate holds. The internal and autoregressive frontiers K_{internal} and K_{AR} are task- and protocol-specific projections of a generally non-rectangular competence region. A complete report distinguishes Bayes solvability, representability, learnability, reliable acquisition, coverage, composition frontier, recurrence frontier, and closure signature.

Let Q be the latent event that a reusable procedure was acquired and S the event of evaluation success. Then

$$P(S) = P(Q)P(S | Q) + P(\neg Q)P(S | \neg Q).$$

The shorthand $P(S) \approx P_{\text{acquire}}P(S | Q)$ is valid only when shortcut success in the $\neg Q$ class is negligible. Because Q is not directly observed, controls and interventions identify a useful mixture model rather than a unique psychological state.

Proposition 2 (Local-error accumulation). *If an execution requires T transitions and, conditional on every previously correct state, each transition is correct independently with probability $1 - \epsilon$, then exact trajectory success is $(1 - \epsilon)^T \leq e^{-\epsilon T}$.*

Independence is a diagnostic baseline, not a default model: correlated errors, recovery, verification, or history-dependent attention can produce slower or faster decay. Reporting both local and sequence accuracy tests which regime applies.

Proposition 3 (Finite evidence does not establish closure). *For any finite tested set $K \leq K_{\text{test}}$, there exist a closed procedure and a bounded lookup machine that agree on every observation and diverge for some $K > K_{\text{test}}$.*

Construct the lookup machine to copy the closed procedure on the finite tested domain and return a fixed wrong answer outside it. Thus empirical evidence can support closure signatures and falsify particular bounded hypotheses, but cannot prove the universal quantifier.

No empirical resource monotonicity is assumed. Although a larger architecture may contain a smaller computation representationally, its optimizer, implicit bias, attention geometry, and finite-data basin differ. Hence $P_{\text{acquire}}(L + 1) \geq P_{\text{acquire}}(L)$ is an empirical hypothesis, not an axiom.

Failure labels should include:

Bayes-impossible, missing fact, operator, composition, recurrence, counter/control,
termination, search, context overflow, accumulated local error, acquisition-basin failure.

6 Relation to classical computation theory

Classical automata theory classifies languages and machines using memory, state transition, stack access, and recursion. Circuit complexity classifies computations by size, depth, fan-in, and gate type. Formal Transformer theory similarly derives expressivity bounds under assumptions about attention, precision, positions, and uniformity.

The present program adds an empirical learned-machine dimension. A construction proving that a Transformer *can* implement an operation establishes representability, not that SGD will find it from a finite generator. Conversely, a bounded empirical failure does not establish architectural impossibility.

The desired bridge is

machine resources + task structure + training distribution $\longrightarrow$ acquisition and execution frontier
--

7 Related work

Table 2 locates the learned-machine layer relative to representative theory. The labels summarize theorem assumptions rather than asserting equivalence between models.

Work	Machine	Assumptions emphasized	Result type	Gap addressed here
Hahn (2020)	fixed attention	bounded depth; formal-language setting	limitation/sensitivity	training and finite acquisition
Hao et al. (2022)	fixed hard attention	circuit assumptions, precision	AC ⁰ -related bounds	empirical competence regions
Pérez et al. (2021)	recurrent/encoded Transformer	explicit precision/encoding assumptions	Turing-completeness construction	probability of learning construction
Strobl et al. (2024)	multiple variants	positions, precision, uniformity	expressivity survey	common reporting schema
Li et al. (2024)	autoregressive CoT	serial intermediate tokens	expressivity separation	acquired finite K_{AR}
Joshi et al. (2025)	autoregressive learner	time-invariant generator	learning/sample analysis	seed and generator acquisition
Bavandpour et al. (2025)	hard-attention CoT	scratchpad assumptions	length lower bounds	measured closure signatures
Huang et al. (2025)	trained recognizer	regular-language distributions	training dynamics	broader task/resource phase diagram

Table 2: Representative related-work matrix. Precision, positional assumptions, and uniformity must be read from each theorem; no row transfers automatically to the finite models studied here.

Formal-language and circuit-complexity analyses have established limitations and capabilities of fixed-depth Transformer variants under explicit assumptions [1, 2]. Strobl et al. survey this literature and emphasize that conclusions depend critically on precision, positional encodings, attention definitions, and uniformity [5]. Universality and Turing-completeness constructions establish strong representability under richer assumptions [4, 3], but do not answer finite-data acquisition probability.

Recent chain-of-thought theory is especially relevant. Li et al. show that intermediate autoregressive steps can enable inherently serial computations inaccessible to shallow one-pass models under their assumptions [6]. Joshi et al. formalize learning with an iterated time-invariant next-token generator and analyze sample/computational complexity [7]. Bavandpour et al. derive lower bounds on scratchpad length for algorithmic problems in hard-attention Transformers [8]. Huang et al. study training dynamics for regular-language recognition [9]. These works motivate treating generated tokens as a computational resource rather than merely explanatory prose.

Our emphasis is different: measure learned finite-machine frontiers, acquisition probability, information sufficiency, and transitions between one-pass, recurrent, contextual, and harnessed execution in tiny models where causal inspection is feasible.

8 Empirical anchors from the series

These anchors occur after the framework because none is needed for its definitions or propositions. They are bounded observations, not universal claims.

Paper 0.5: one-pass refinement. Across three controlled generators, depth improves predictive-class organization, usually non-monotonically. Signal grows faster than fluctuation; absolute margin variance does not contract. Local-attention reachability is necessary but insufficient, and head effects are distributed and non-additive. This anchors Axes 1–2 while falsifying simple monotone-depth, contraction, and stable-head-role stories.

Paper 0.6: competence and allocation. Bounded structured attribute composition is a positive generator-specific result, but the original depth-general predicate phase passes no three-seed gate. A later designed capacity diagnostic finds that depth 8 makes `isAncestor` seed-stable on easy and medium tiers (mean/worst .935/.889 and .928/.882), but not hard (.869/.759); four other predicates remain far below competence. This is a finite, allocation-sensitive membership frontier, not general predicate closure. Its planned training-budget control is not reported as a result.

Paper 0.8: information versus acquisition. The D4 permutation-completion task is information-sufficient: under its declared prior, context reduces target uncertainty from $\log_2 3$ to zero. Yet none of five Pareto-small

candidates passes all three seeds. In contextual-copy calibration, one competent checkpoint exhibits pair binding and later value routing, while fresh replications miss the 95% three-seed gate. At 1,024 episodes, seeds 11 and 23 reach 100%, whereas seed 37 moves from 52.1% at nominal budget to 81.2% at $2\times$ and 60.4% at a targeted $4\times$ run. This directly motivates separating information sufficiency, physical mechanism after acquisition, and reliable acquisition.

Papers 0.7, 0.85, and 0.9: pending gates. Paper 0.7 now has exact CPU-validated propositional and unification generators, but no formal-model frontier is yet a finding. Paper 0.85 will compare one-token and unconstrained multi-token machines; Paper 0.9 will compare scratchpad, proto-tool, and harness transitions. Their planned comparisons are hypotheses, not empirical anchors.

9 Core falsifiable conjectures

- C1.** Reliable acquisition of a local operator does not imply reliable acquisition of its finite compositions.
- C2.** Many iterative task families exhibit a finite one-pass composition frontier for a fixed trained Transformer.
- C3.** Multi-token state generation substantially expands the measured composition frontier for tasks admitting a reusable local transition.
- C4.** After local competence, long-recurrence failure increasingly reflects state representation, termination, counters, and accumulated error.
- C5.** Supplying facts/rules in context separates factual memory failure from procedural failure.
- C6.** For some tasks, explicit harnesses greatly improve acquisition/execution reliability without changing what an ideal autoregressive Transformer could represent in principle.

10 Open problems

The framework sharpens eight questions. Which local operators have high acquisition probability at a fixed scale and distribution? Which resource interactions move one-pass composition frontiers? When does autoregressive state produce a stable train-short/test-long signature rather than a longer bounded repertoire? Which tasks genuinely require counters, multiple state variables, stacks, substitutions, or visited-state memory under a specified interface? Does context provide missing facts, choose among stored tasks, estimate parameters, or construct a transition? When does a harness enlarge the semantic machine rather than stabilize an already representable trajectory? Can empirical acquisition boundaries be connected to circuit or automata classes without discarding precision and uniformity assumptions? Finally, what generator and curriculum properties favor a reusable transition over memorized finite coverage?

11 Conclusion

A theory of computation for LLMs should distinguish what a model can represent, what training can reliably acquire, what one pass can compose, what autoregressive recurrence can close, what context contributes as information, and what external execution adds as either new resources or reliability. This paper proposes that classification as the organizing framework for controlled experiments on learned Transformer machines.

References

- [1] M. Hahn. Theoretical Limitations of Self-Attention in Neural Sequence Models. *TACL*, 2020.
- [2] Y. Hao, D. Angluin, and R. Frank. Formal Language Recognition by Hard Attention Transformers: Perspectives from Circuit Complexity. *TACL*, 2022.

- [3] J. Pérez, P. Barceló, and J. Marinkovic. Attention is Turing Complete. *JMLR*, 2021.
- [4] C. Yun et al. Are Transformers Universal Approximators of Sequence-to-Sequence Functions? *ICLR*, 2020.
- [5] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. What Formal Languages Can Transformers Express? A Survey. *TACL*, 2024.
- [6] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of Thought Empowers Transformers to Solve Inherently Serial Problems. arXiv:2402.12875, 2024.
- [7] N. Joshi et al. A Theory of Learning with Autoregressive Chain of Thought. *COLT*, 2025.
- [8] A. A. Bavandpour et al. Lower Bounds for Chain-of-Thought Reasoning in Hard-Attention Transformers. *ICML*, 2025.
- [9] R. Huang, Y. Liang, and J. Yang. How Transformers Learn Regular Language Recognition: A Theoretical Study on Training Dynamics and Implicit Bias. *ICML*, 2025.